

# Probar servicios con MockMvc

## *3. Probar servicios con MockMvc*

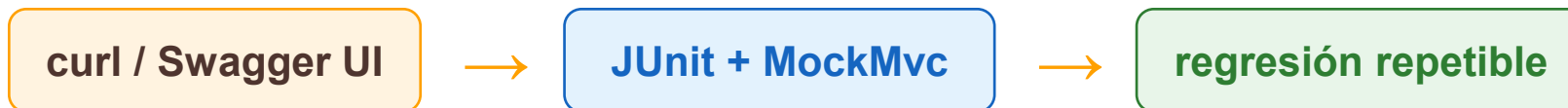
## Pasar de “lo pruebo a mano” a “lo compruebo automáticamente”

### Prueba manual

- Abrir Swagger UI o escribir curl
- Repetir los mismos pasos tras cada cambio
- Depender de que alguien recuerde qué comprobar

### Test automatizado

- La comprobación queda escrita como código
- Se ejecuta en segundos y siempre igual
- Puede lanzarse localmente o desde CI



**Idea clave: el test convierte una expectativa en una comprobación ejecutable.**

## Anatomía de un test automatizado

El patrón **Arrange** → **Act** → **Assert**

### 1. Arrange (Preparar)

Preparas el objeto y los datos de prueba.



```
Java
Calculadora calc = new Calculadora();
int a = 2;
int b = 3;
```

### 2. Act (Actuar)

Ejecutas la acción que quieres probar.



```
Java
int resultado = calc.sumar(a, b);
```

### 3. Assert (Comprobar)

Verificas que el resultado es el esperado.



```
Java
assertEquals(5, resultado);
```

## Cómo leer un test

#### esperado

El valor que debería obtenerse.

#### actual

El valor que realmente se ha obtenido.

#### comparación

Se comparan ambos valores.



#### Ejemplo de fallo de test:

Expected: **6**  
Actual: **5**

Si no coinciden, el test falla y te da una pista del problema.



## Idea clave

Un buen test automatizado suele responder a tres preguntas:

- 1 ¿qué **preparo**?
- 2 ¿qué **ejecuto**?
- 3 ¿qué **compruebo**?

# Dependencia real vs. mock

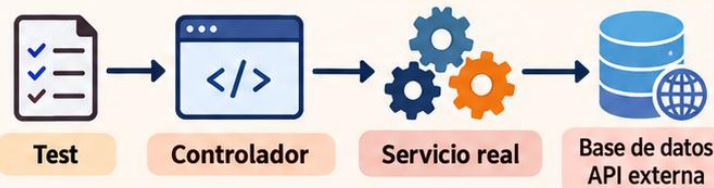


## Dependencia real vs mock

Qué cambia en una prueba y por qué usamos mocks



### Dependencia real



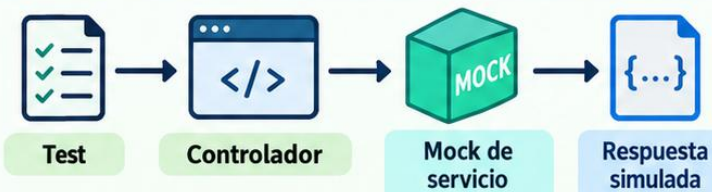
- 1 Se ejecutan más piezas del sistema
- 2 Puede depender de datos, red o configuración
- 3 Más coste y más variabilidad
- 4 Útil cuando quieres probar integración real

#### Ejemplo

```
@Autowired
LibroService service;
```

**Más realismo,  
menos aislamiento**

### Mock



- 1 Aíslas la unidad que quieres comprobar
- 2 Controlas exactamente la respuesta
- 3 La prueba es más rápida y predecible
- 4 Ideal para probar comportamiento del controlador

#### Ejemplo

```
@MockBean
LibroService service;

when(service.buscarPorId(1L))
    .thenReturn(libro);
```

**Más control,  
más aislamiento**



#### Idea clave



**Dependencia real:**  
comprueba colaboración real  
entre componentes

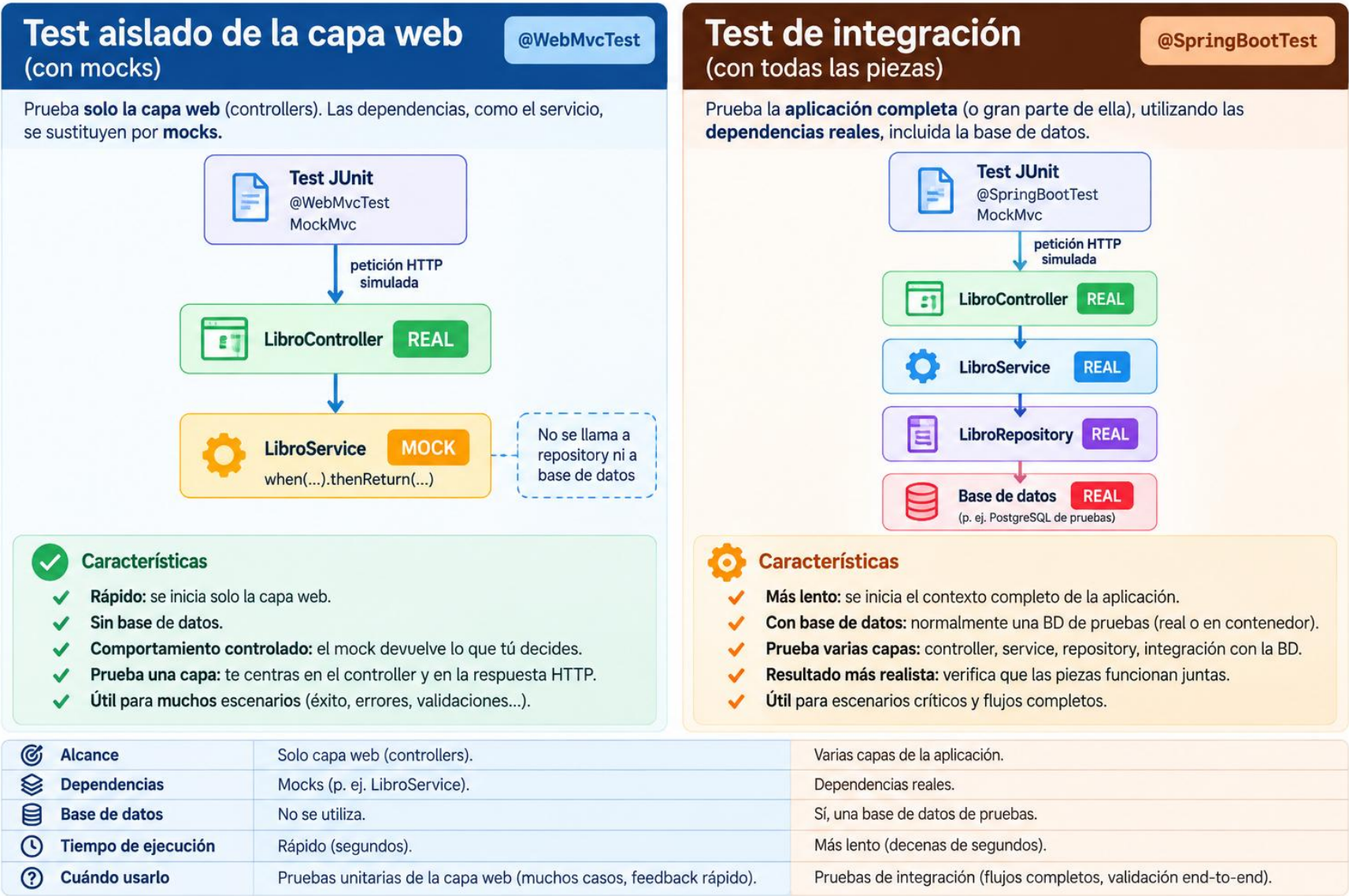


**Mock:**  
simula colaboradores  
para centrar la prueba



En **MockMvc** es frecuente  
mockear servicios para centrarse  
en HTTP, JSON y respuestas

# Test aislado vs. test de integración



# MockMvc: un cliente HTTP programable

## Tres formas de enviar una petición a la misma API

**curl**

Manual · terminal

**Swagger UI**

Manual · navegador

**MockMvc**

Automático · Java

### ¿Qué aporta MockMvc?

- No necesita un servidor HTTP real
- Permite expresar la petición con código
- Comprueba estado, cabeceras y JSON
- Es rápido, repetible y automatizable

**No hay puerto 8080 real**

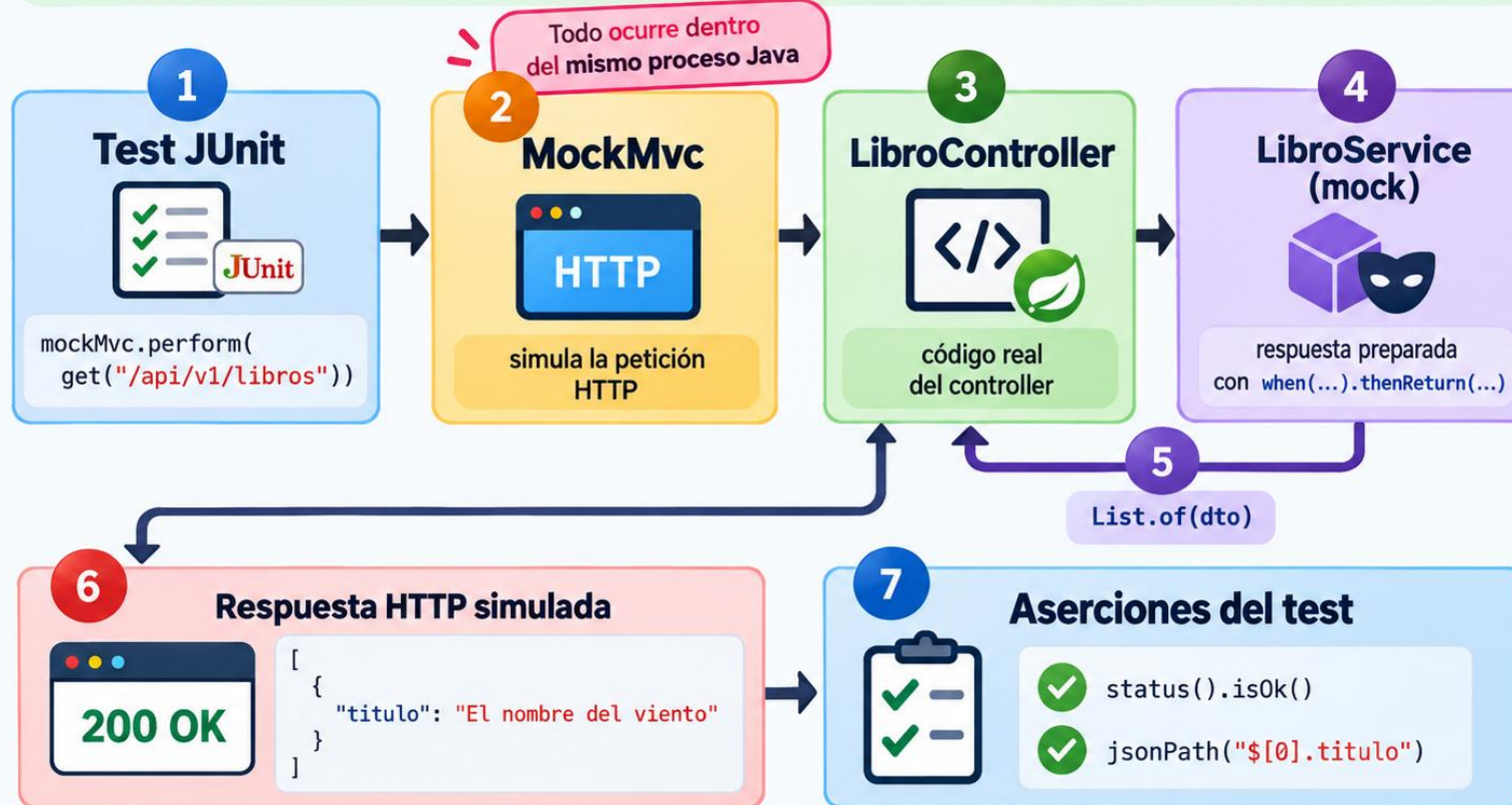
La petición se procesa dentro del proceso Java.

**MockMvc prueba la capa web sin depender de red externa.**

# Cómo viaja una petición en MockMvc

## Cómo viaja una petición en MockMvc

Del test JUnit al controller y de vuelta, sin arrancar un servidor HTTP real.



**Idea clave**



No hay puerto 8080 real



Se prueba la capa web de forma aislada



La prueba es rápida y repetible

# Primer test real con @WebMvcTest

```
@WebMvcTest(LibroController.class)
@AutoConfigureMockMvc(addFilters = false)
class LibroControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockitoBean
    private LibroService libroService;

    @Test
    void getAll_DebeDevolverListaDeLibros() throws Exception {
        var dto = new LibroResponseDTO(1L, "El nombre del viento", ...);

        when(libroService.findAll()).thenReturn(List.of(dto));

        mockMvc.perform(get("/api/v1/libros"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.titulo")
                .value("El nombre del viento"));
    }
}
```

## Qué debes localizar

- @WebMvcTest: capa web
- @MockitoBean: dependencia falsa
- when(...): preparar
- perform(...): actuar
- andExpect(...): afirmar

**El test conserva el controller real y controla el service.**

# Anatomía de un test con @WebMvcTest

## Anatomía de un test con @WebMvcTest y MockMvc

Así se prueba un endpoint del LibroController de forma aislada, sustituyendo el servicio por un mock.

```
1 1 @WebMvcTest(LibroController.class)
2   class LibroControllerTest {
3
4     2 @Autowired
5       private MockMvc mockMvc;
6
7     3 @MockitoBean
8       private LibroService libroService;
9
10    @Test
11    void getAll_DebeDevolverListaDeLibros() throws Exception {
12
13      4 // Preparar: qué debe devolver el mock cuando se llame al servicio
14      when(libroService.findAll()).thenReturn(List.of(
15        new LibroResponseDTO(1L, "El principito", "Antoine de Saint-Exupéry", 1943),
16        new LibroResponseDTO(2L, "Clean Code", "Robert C. Martin", 2008)
17      ));
18
19      5 // Actuar: simular una petición HTTP GET al endpoint
20      mockMvc.perform(get("/api/v1/libros")
21        .accept(MediaType.APPLICATION_JSON))
22
23      6 // Afirmar: comprobar la respuesta que devuelve el controller
24      .andExpect(status().isOk())
25      .andExpect(content().contentType(MediaType.APPLICATION_JSON))
26      .andExpect(jsonPath("$.titulo").value("El principito"))
27      .andExpect(jsonPath("$.autor").value("Clean Code"));
28    }
29 }
```

### 1 @WebMvcTest(...)

- Carga solo la capa web (controllers).
- No arranca el servicio, el repositorio ni la base de datos.
- Ideal para probar los endpoints de forma aislada.

### 2 MockMvc

- Cliente HTTP programable que simula peticiones.
- Permite indicar la ruta, el verbo, el cuerpo, cabeceras, etc.
- Devuelve una respuesta que podemos comprobar.

### 3 @MockitoBean

- Sustituye el LibroService real por un mock.
- Spring se encarga de crear el mock e inyectarlo en el controller.
- Permite decidir qué devuelve cuando se le llama.

### 4 Preparar (when ... thenReturn ...)

- Definimos el comportamiento del mock.
- En este caso, cuando se llame a findAll() devolverá una lista de libros concreta, sin necesidad de base de datos.

### 5 Actuar (mockMvc.perform ...)

- Simulamos una petición HTTP al endpoint real.
- Indicamos el verbo (get), la ruta (/api/v1/libros) y el tipo de respuesta que esperamos (accept).

### 6 Afirmar (andExpect ...)

- Comprobamos el código de estado (isOk → 200).
- Verificamos el tipo de contenido (JSON).
- Comprobamos datos concretos de la respuesta con jsonPath.



#### Preparar

Definir qué debe devolver el mock del servicio

1



#### Actuar

Simular la petición HTTP con MockMvc

2



#### Afirmar

Comprobar el código de estado y el contenido de la respuesta

3



#### Resultado

Test rápido, sin base de datos, y centrado en la capa web

4

# Tres situaciones habituales en un controller

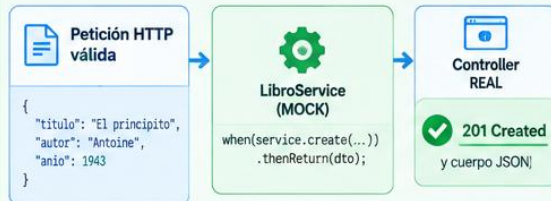
## Tres formas de responder en un test de controlador

Según cómo prepares (o no prepares) el mock del servicio, el controller puede devolver un resultado de éxito, un error de validación o un error de negocio.

### 1 Respuesta de éxito

thenReturn(...)

El mock devuelve un dato y el controller responde correctamente.



#### En el test

```
// Preparar: el mock devuelve un LibroResponseDTO
when(libroService.create(any(LibroCreateDTO.class)))
    .thenReturn(new LibroResponseDTO(
        1L, "El principito", "Antoine", 1943
    ));

// Actuar y afirmar
mockMvc.perform(post("/api/v1/libros")
    .contentType(MediaType.APPLICATION_JSON)
    .content(jsonBodyValido))
    .andExpect(status().isCreated())
    .andExpect(jsonPath("$.titulo").value("El principito"));
```

#### Resultado en la respuesta

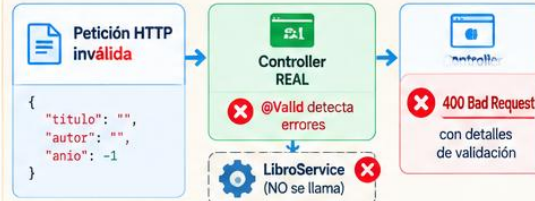
Código HTTP **201 Created** Cuerpo JSON (fragmento)

```
{
  "id": 1,
  "titulo": "El principito",
  "autor": "Antoine",
  "anio": 1943
}
```

### 2 Error de validación

sin preparar el mock

El DTO no cumple las validaciones y el controller devuelve 400. El servicio no se llega a llamar.



#### En el test

```
// No preparamos nada en el mock: no se va a llamar
mockMvc.perform(post("/api/v1/libros")
    .contentType(MediaType.APPLICATION_JSON)
    .content(jsonBodyInvalido))
    .andExpect(status().isBadRequest())
    .andExpect(jsonPath("$.errors").isArray())
    .andExpect(jsonPath("$.errors[0].field").value("titulo"));

// Comprobamos que efectivamente no se llama al servicio
verifyNoInteractions(libroService);
```

#### Resultado en la respuesta

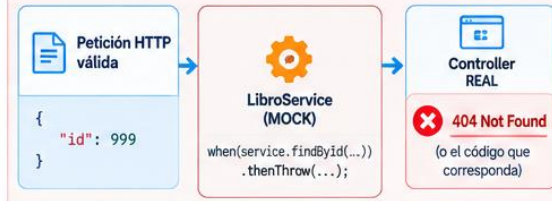
Código HTTP **400 Bad Request** Cuerpo JSON (fragmento)

```
{
  "errors": [
    {
      "field": "titulo",
      "message": "El título no puede estar vacío"
    }
  ]
}
```

### 3 Error de negocio

thenThrow(...)

El mock lanza una excepción (por ejemplo, recurso no encontrado) y el controller devuelve el código de error correspondiente.



#### En el test

```
// Preparar: el mock lanza la excepción que usamos en el proyecto
when(libroService.findById(999L))
    .thenThrow(new RecursoNoEncontradoException("Libro", 999L));

// Actuar y afirmar
mockMvc.perform(get("/api/v1/libros/999"))
    .andExpect(status().isNotFound())
    .andExpect(jsonPath("$.message").value("Libro no encontrado"));
```

#### Resultado en la respuesta

Código HTTP **404 Not Found** Cuerpo JSON (fragmento)

```
{
  "message": "Libro no encontrado",
  "recurso": "Libro",
  "id": 999
}
```

#### i Ideas clave

✓ El mock te permite decidir cómo se comporta el servicio en cada escenario.

✓ El controller responde con el código y el cuerpo adecuados según la situación.

✓ Puedes comprobar, además, si el servicio se ha llamado o no (verify, verifyNoInteractions, ...).

✓ Estos tres casos cubren los escenarios más habituales en los tests de la capa web.

# Actividad 1.3: comprobar lo que ya has documentado

No construyes endpoints nuevos: escribes comprobaciones automáticas sobre los existentes.

## VideojuegoController

- casos 200 / 201
- validación 400
- recurso inexistente 404
- JSON esperado

## EstudioController

- casos 200 / 201
- validación 400
- recurso inexistente 404
- JSON esperado

Documentación OpenAPI



Tests MockMvc



evidencia ejecutable

**La prueba debe demostrar que los códigos que has documentado son realmente los que devuelve la API.**

# Después de MockMvc: ampliar el alcance

Cada tipo de prueba responde a una pregunta diferente

## @WebMvcTest

- ¿Responde bien el controller?
- Dependencias mockeadas
- Feedback rápido

## Test de integración

- ¿Funcionan juntas varias capas?
- Dependencias reales
- BD de prueba / Testcontainers

## Actividad 1.4

- Concurrencia y tiempos
- Observabilidad
- Actuator

A más aislamiento, más velocidad y control. A más integración, más realismo y más coste.

**No hay un único “mejor” tipo de test: elige el alcance que responda a la pregunta que quieres comprobar.**

## Lo mínimo que debe quedar claro

- ✓ Un test automatizado convierte una expectativa en una comprobación repetible.
- ✓ Arrange → Act → Assert organiza qué preparas, qué ejecutas y qué afirmas.
- ✓ Un mock sustituye dependencias para aislar la parte que quieres probar.
- ✓ MockMvc prueba controllers sin arrancar un servidor HTTP real.
- ✓ @WebMvcTest + @MockitoBean + perform/andExpect forman el patrón básico de esta sesión.
- ✓ thenReturn, validación previa y thenThrow cubren escenarios distintos.

**Próxima parada: medir comportamiento bajo carga y observar la aplicación con Actuator.**